

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA1017	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	LOSHINI.L	Reg. No.:	192571015
Date:	13/8/26	Duration:	60 minutes

Code of Conduct

I (Name/ Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Annexure A – Git & Docker Technical Assignment

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, prepare a **Git & Docker Technical Assignment** by applying version control and containerization concepts. Your report should include appropriate diagrams, screenshots, commands, and explanations wherever applicable.

Assignment Questions

- 1. Problem Analysis**
 - Explain the assigned problem statement and identify the project objectives, functional requirements, and expected outcomes.
- 2. Project Structure Design**
 - Design and explain the folder structure of your project repository.
 - Justify the purpose of each major folder and file included in the repository.
- 3. Git Repository Initialization**
 - Create a Git repository for the project.
 - Explain the steps involved in repository initialization and describe the purpose of the essential Git files.
- 4. Git Branching Strategy**
 - Design a suitable branching strategy (e.g., Main, Development, Feature, Release, Hotfix).
 - Justify why the selected branching model is appropriate for the assigned project.
- 5. Version Control Workflow**
 - Demonstrate the Git workflow by performing meaningful commits, branch creation, merging, conflict resolution (if applicable), and repository synchronization.
 - Explain the purpose of each Git operation performed.
- 6. Commit History Analysis**
 - Present the commit history of the project.

- Explain how commit messages follow good version control practices and contribute to project maintenance.
- 7. Docker Environment Design**
 - Explain why Docker is suitable for the assigned project.
 - Identify the application components that require containerization.
- 8. Dockerfile Development**
 - Design and explain the Dockerfile required to containerize the application.
 - Describe the purpose of each instruction used in the Dockerfile.
- 9. Docker Compose Design**
 - Develop a Docker Compose configuration for the project.
 - Explain the services, networking, volumes, environment variables, and dependencies used.
- 10. Container Deployment and Testing**
 - Demonstrate the execution of the application using Docker containers.
 - Explain the testing procedure and provide evidence that the application runs successfully.
- 11. Benefits of Git and Docker**
 - Discuss how Git and Docker improve collaboration, version management, portability, deployment, scalability, and maintainability for your assigned project.
- 12. Challenges and Improvements**
 - Identify the challenges encountered during implementation.
 - Suggest possible improvements and future enhancements for the Git workflow and Docker deployment.

Expected Deliverables

- Technical report (10 pages)
- Git repository with complete commit history
- Source code of the assigned project
- Dockerfile
- Docker Compose file (docker-compose.yml)
- Screenshots of Git operations and Docker execution
- Architecture and workflow diagrams

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Project Design(20%)	Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification.	Good analysis and project structure with minor omissions.	Basic analysis and repository structure with limited justification.	Incomplete or unclear analysis and project organization.
Git Repository & Branching Strategy(20%)	Repository initialized correctly with appropriate branching strategy, clear workflow, and proper repository organization.	Repository and branching strategy are mostly correct.	Basic Git setup with limited branching implementation.	Incorrect or incomplete Git repository and branching strategy.

Version Control Workflow & Commit History(10%)	Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution, and professional commit messages with clear history.	Most Git operations demonstrated correctly with good commit history.	Limited Git workflow and commit practices.	Poor workflow and inadequate commit history.
Docker Implementation(20%)	Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies.	Functional Docker implementation with minor issues.	Basic Docker implementation with limited explanation.	Incomplete or incorrect Docker implementation.
Deployment, Testing & Results(15%)	Application successfully deployed and tested with appropriate screenshots and evidence of successful execution.	Deployment completed with minor issues.	Partial deployment and testing demonstrated.	Deployment unsuccessful or insufficient evidence.
Documentation, Challenges & Future Enhancements(15%)	Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future improvements.	Good documentation with minor omissions.	Basic documentation with limited discussion.	Poor documentation and insufficient analysis.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

1. Problem Analysis

1.1 Problem Statement

Municipal solid waste today is largely sorted by hand, which is slow, inconsistent, and expensive to operate at scale. Poor segregation at source and at the facility means recyclable material ends up in landfills, recycling yields stay low, and disposal costs and environmental pollution keep rising. The assigned project addresses this by building an AI-powered waste segregation and recycling automation system that uses computer vision to recognise waste categories in real time, drives a robotic sorting mechanism to physically separate the material, tracks recycling statistics over time, and produces management reports that facility operators can act on.

1.2 Project Objectives

- Automatically classify waste items (e.g., plastic, paper, metal, glass, organic, e-waste) using a computer-vision model.
- Drive a robotic sorting mechanism (arm / diverter on a conveyor) based on the classification result.
- Continuously log sorting events to build recycling statistics per category, per shift, and per facility.
- Generate periodic (daily/weekly/monthly) waste-management and recycling-efficiency reports.
- Provide a web dashboard for facility operators and municipal authorities to monitor operations in real time.
- Improve recycling efficiency and reduce landfill load and environmental pollution.

1.3 Functional Requirements

- Image capture pipeline that streams frames from a conveyor-mounted camera to the classification service.
- A trained CNN-based classifier that returns a waste category and confidence score per item.
- A robotic controller service that converts classification output into actuator commands (arm angle / diverter gate).
- A backend API that persists classification and sorting events, and exposes them to the frontend.
- A relational database that stores items sorted, categories, timestamps, and facility metadata.
- A reporting module that aggregates data into recycling-efficiency and waste-composition reports.
- A role-based dashboard (operator / administrator) showing live status, historical trends, and alerts.

1.4 Expected Outcomes

- Higher recycling accuracy compared with manual sorting, with measurable reduction in contamination rate.
- Lower operational cost per tonne of waste processed due to reduced manual labour dependency.
- Real-time visibility into sorting performance through the dashboard.
- Automated, auditable reports that support municipal decision-making and compliance reporting.

2. Project Structure Design

The system is designed as a set of independently deployable services — computer-vision inference, robotic control, backend API, and frontend dashboard — coordinated through a shared message broker and database.

This maps naturally onto a monorepo with one top-level folder per service, which keeps Docker builds, CI pipelines, and Git history scoped per component while still allowing atomic cross-service commits when an interface changes.

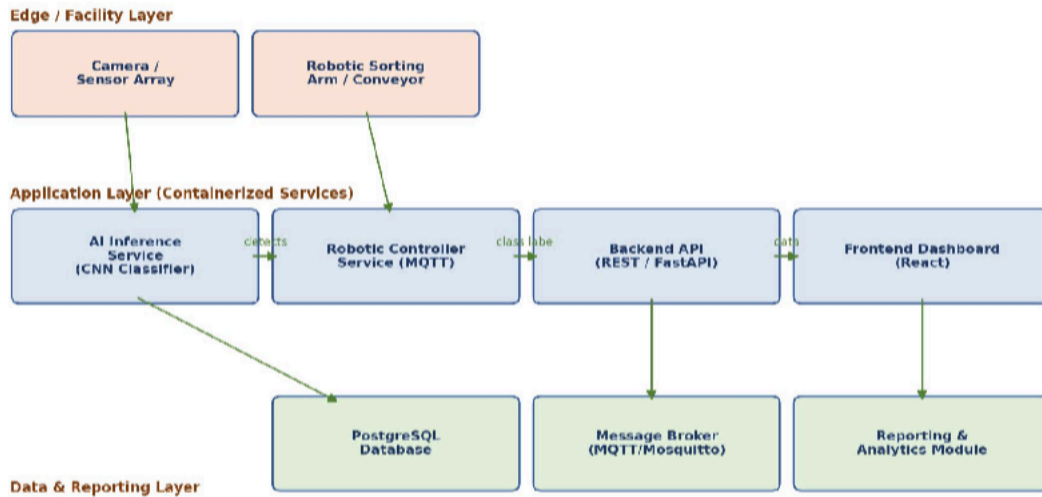
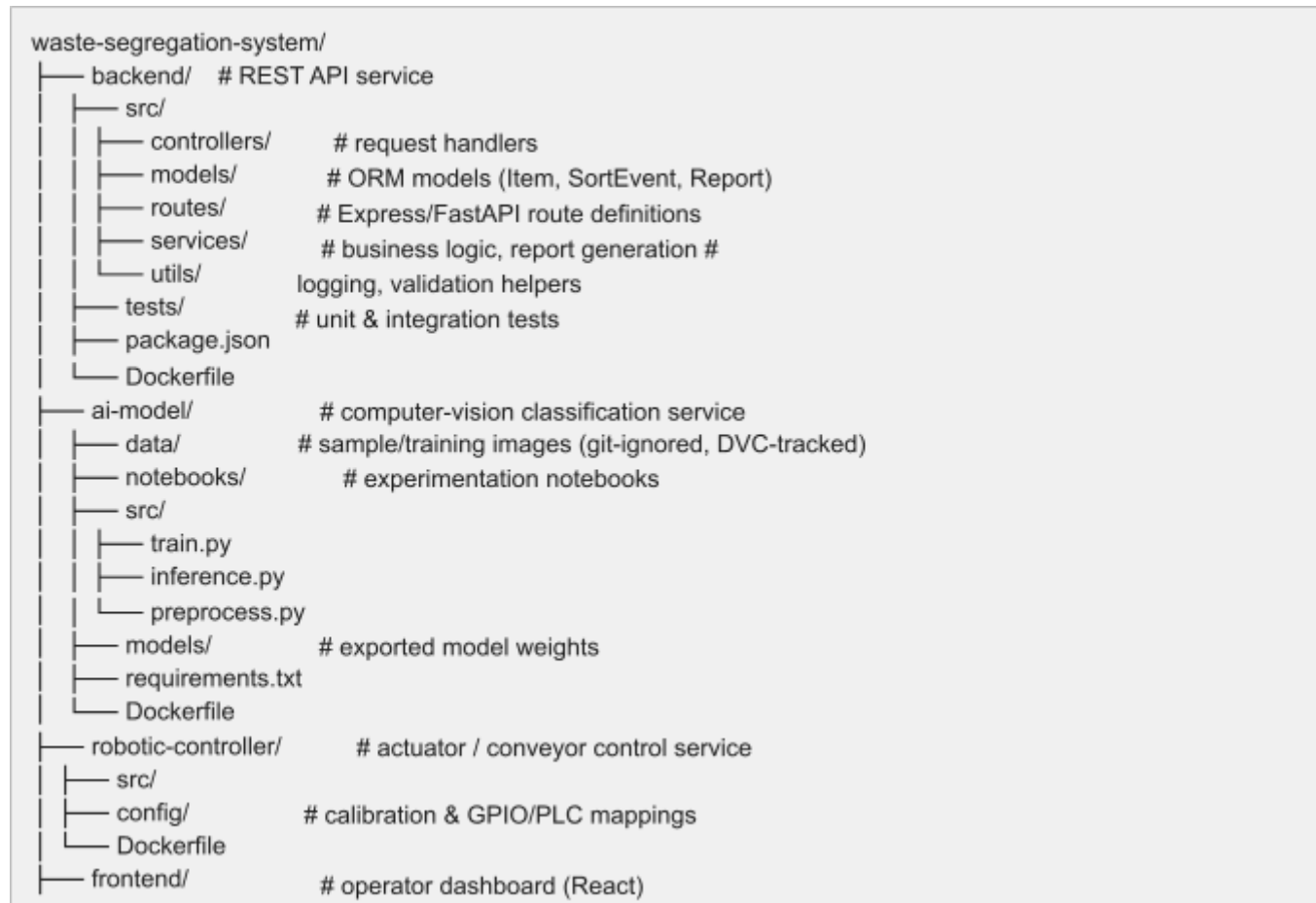


Figure 1: High-level system architecture — edge, application, and data/reporting layers



```

|
├── src/
│   ├── components/
│   ├── pages/
│   └── services/      # API clients
├── public/
├── package.json
├── Dockerfile
├── database/
│   └── init.sql      # schema + seed data
├── docs/             # architecture & workflow diagrams, report
├── .github/workflows/ci.yml    # CI pipeline
├── docker-compose.yml
├── .env.example
├── .gitignore
└── README.md

```

2.1 Purpose of Major Folders and Files

Folder / File	Purpose
backend/	Hosts the REST API that receives classification events, persists them, and serves data to the frontend and reporting module.
ai-model/	Isolates the computer-vision training and inference code and its heavy ML dependencies from the rest of the stack.
robotic-controller/	Encapsulates hardware-facing logic (actuator commands, calibration) so it can be developed and tested independently of the AI model.
frontend/	React application giving operators a live view of sorting activity and historical reports.
database/init.sql	Defines the schema (items, sort_events, reports, users) so a fresh environment can be bootstrapped consistently.
docs/	Central location for architecture diagrams, the branching-strategy diagram, and this report, kept in version control alongside the code.
.github/workflows/ci.yml	Runs linting, unit tests, and Docker builds automatically on every push and pull request.
docker-compose.yml	Declares and wires together all containers (backend, ai-model, robotic-controller, frontend, database, broker) for local and staging deployment.
.env.example	Documents required environment variables (DB credentials, broker host, model path) without committing real secrets.

Folder / File	Purpose
.gitignore	Excludes build artefacts, dependency folders, and large binary files (e.g. trained model weights, datasets) from Git.

3. Git Repository Initialization

The repository was initialised locally and then linked to a remote (GitHub) so that the four service teams — AI/ML, backend, robotics, and frontend — could collaborate against a single source of truth.

```
$ mkdir waste-segregation-system && cd waste-segregation-system
$ git init
$ git branch -M main

# create baseline files
$ touch README.md .gitignore .env.example docker-compose.yml
$ mkdir backend ai-model robotic-controller frontend database docs

$ git add .
$ git commit -m "chore: initialise repository structure for waste segregation system"

# link to remote
$ git remote add origin https://github.com/<org>/waste-segregation-system.git
$ git push -u origin main
```

3.1 Purpose of Essential Git Files

File	Purpose
.git/	Hidden directory created by 'git init'; stores the full object database, refs, and configuration for the repository.
.gitignore	Prevents dependency folders (node_modules/, _pycache_/), environment files (.env), and large model/dataset binaries from being tracked.
README.md	Documents project purpose, setup instructions, and how to run the system with Docker Compose.
.env.example	Template listing required environment variables without exposing real credentials.
LICENSE (optional)	Declares the terms under which the source code may be reused.

4. Git Branching Strategy

A Gitflow-style branching model was chosen because the project has clearly separated, parallel workstreams (AI model, robotic controller, backend, frontend), a distinct staged rollout to physical facilities (release branches), and a need to patch production sorting stations quickly without disturbing in-progress feature work (hotfix branches).

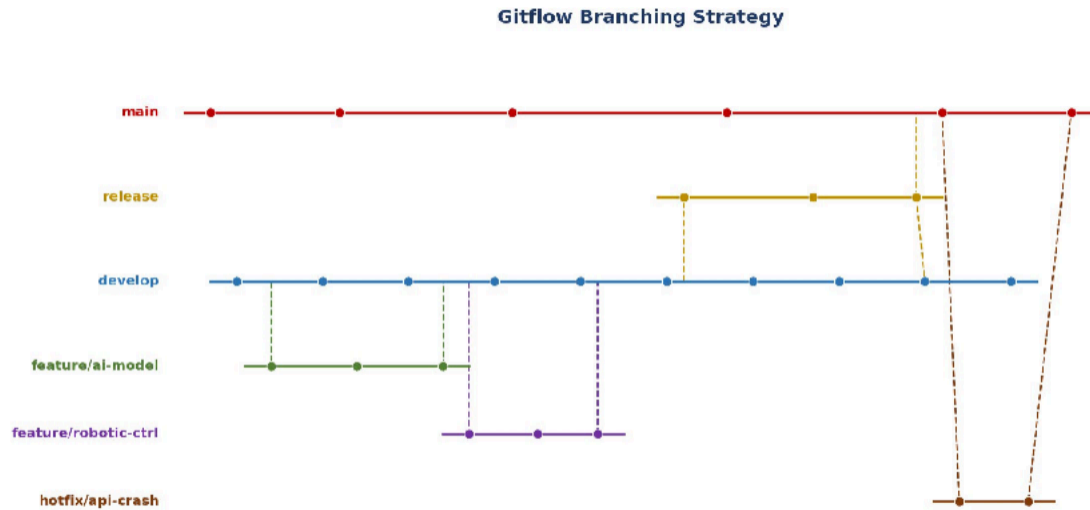


Figure 2: Gitflow branching strategy adopted for the project

Branch	Purpose
main	Always reflects the production-ready state deployed at live facilities; every commit here is tagged with a release version.
develop	Integration branch where completed features from all four teams are merged and tested together before a release.
feature/*	One branch per unit of work, e.g. feature/ai-model, feature/robotic-ctrl, feature/report-dashboard; branched from and merged back into develop.
release/*	Cut from develop when preparing a facility rollout; only bug fixes and documentation changes are allowed here before merging into both main and develop.
hotfix/*	Branched directly from main to fix a critical production issue (e.g. sorting-arm mis-trigger) and merged back into both main and develop.

Justification: separating AI/ML, robotics, backend, and frontend work into feature branches lets each team iterate without blocking the others, develop provides a single integration point to catch cross-service issues early, release branches let the facility-deployment team stabilise a version before it reaches physical hardware, and hotfix branches guarantee that a live sorting line can be patched within minutes without waiting for the next full release cycle.

5. Version Control Workflow

The following sequence demonstrates the day-to-day workflow used while building the waste-classification feature, including branch creation, meaningful commits, a merge, conflict resolution, and synchronisation with the remote.

```
# 1. create a feature branch from develop
$ git checkout develop
$ git pull origin develop
$ git checkout -b feature/ai-model

# 2. meaningful, incremental commits
$ git add ai-model/src/preprocess.py
$ git commit -m "feat(ai-model): add image resize and normalisation pipeline"

$ git add ai-model/src/train.py
$ git commit -m "feat(ai-model): add CNN training script with data augmentation"

$ git add ai-model/src/inference.py
$ git commit -m "feat(ai-model): add inference endpoint returning category + confidence"

# 3. keep the branch in sync with develop
$ git fetch origin
$ git rebase origin/develop

# 4. push and open a pull request
$ git push -u origin feature/ai-model
```

5.1 Conflict Resolution Example

A conflict occurred when both the AI-model and backend teams modified requirements.txt (one adding tensorflow, the other adding sqlalchemy) on branches merged close together.

```
$ git checkout develop
$ git merge feature/ai-model
Auto-merging requirements.txt
CONFLICT (content): Merge conflict in requirements.txt Automatic
merge failed; fix conflicts and then commit the result.

# requirements.txt after manual resolution — both dependencies kept
<<<<<<< HEAD
sqlalchemy==2.0.30
=====
tensorflow==2.16.1
>>>>>> feature/ai-model

# resolved to:
sqlalchemy==2.0.30
tensorflow==2.16.1

$ git add requirements.txt
```

```
$ git commit -m "merge: resolve requirements.txt conflict between backend and ai-model"
$ git push origin develop
```

Each git operation serves a distinct purpose: checkout -b isolates new work, incremental commits create a reviewable history, fetch/rebase keeps a branch current with the least merge noise, merge integrates completed work into develop, and the conflict resolution step ensures both teams' dependencies are preserved instead of one silently overwriting the other.

6. Commit History Analysis

```
$ git log --oneline --graph --decorate
* 9f2a1c3 (HEAD -> develop) merge: resolve requirements.txt conflict between backend and ai-model
|\
| * 7b3e5d1 (feature/ai-model) feat(ai-model): add inference endpoint returning category + confidence
| * 4c8f9a0 feat(ai-model): add CNN training script with data augmentation
| * 1a6b2e7 feat(ai-model): add image resize and normalisation pipeline
* | 6d4f0b2 feat(backend): add SortEvent model and persistence layer
|/
* 3e1d8c5 feat(robotic-controller): publish sort command over MQTT on classification
* 0a9b7f4 test(backend): add integration test for /api/sort-events endpoint
* d2c1e6a docs: update README with local Docker Compose instructions
* 8f5a3b9 chore: initialise repository structure for waste segregation system
```

6.1 Commit Message Conventions

Commits follow the Conventional Commits style (type(scope): summary), which keeps the history scannable and lets tooling auto-generate changelogs.

Prefix	Meaning	Example
feat	A new feature	feat(ai-model): add inference endpoint
fix	A bug fix	fix(robotic-controller): correct servo angle offset
docs	Documentation only	docs: update README with setup instructions
test	Adding or updating tests	test(backend): add integration test for sort-events
refactor	Code change with no behaviour change	refactor(backend): extract report service
chore	Tooling / maintenance	chore: initialise repository structure

This convention contributes to maintenance because each entry states its type, the component it touches, and the intent in one line — a reviewer or a future maintainer can scan git log and immediately understand what changed and why, without opening every diff.

7. Docker Environment Design

7.1 Why Docker Is Suitable

- The AI-model service depends on a specific stack (Python, TensorFlow/OpenCV, CUDA libraries) that is easy to break on a shared host; a container pins exact versions.
- Four different runtimes are involved (Python for AI, Node/Python for backend, Node for frontend build, plus the robotic controller) — containers let each use its own base image without conflicts.
- Facilities need identical, repeatable deployments on different physical machines (edge servers at each waste-processing site); an image built once runs the same way everywhere.
- Services must scale independently — during peak intake, more AI-inference replicas can be started without touching the frontend or database.
- Isolation limits the blast radius of a crash: a fault in the robotic-controller container cannot take down the reporting dashboard.

7.2 Components Requiring Containerisation

Component	Reason for Containerisation
ai-model (inference service)	Heavy ML dependencies and, optionally, GPU access; needs a reproducible runtime.
backend (API)	Central service other containers talk to; benefits from consistent Node/Python + DB driver versions.
robotic-controller	Talks to hardware/MQTT; isolating it prevents driver or library conflicts with other services.
frontend (dashboard)	Built once into static assets and served consistently via Nginx in every environment.
database (PostgreSQL)	Standard official image gives a consistent, disposable data layer for dev/staging.
message broker (Mosquitto/MQTT)	Coordinates asynchronous messages between AI, robotic-controller, and backend.

8. Dockerfile Development

8.1 AI Model Service — ai-model/Dockerfile

```
FROM python:3.11-slim

# system libraries required by OpenCV
RUN apt-get update && apt-get install -y --no-
```

```

install-recommends \
  libgl1 libgl2.0-0 && rm -rf
/var/lib/apt/lists/*

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY src/ ./src
COPY models/ ./models

EXPOSE 8001
ENV
MODEL_PATH=/app/models/waste_classifier.h5

CMD ["python", "src/inference.py"]

```

Instruction	Purpose
FROM python:3.11-slim	Selects a minimal, official Python base image to keep the image small.
RUN apt-get install ...	Installs OS-level libraries OpenCV needs for image decoding.
WORKDIR /app	Sets the working directory for all subsequent instructions.
COPY requirements.txt / RUN pip install	Split into two steps so Docker can cache the dependency layer and skip reinstalling when only source code changes.
COPY src/, models/	Adds application code and the trained model weights into the image.
EXPOSE 8001	Documents the port the inference API listens on.
ENV MODEL_PATH	Configures the path to the model file without hard-coding it in source.
CMD	Default command that starts the inference server when the container runs.

8.2 Backend API Service — backend/Dockerfile

```

FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --omit=dev
COPY src/ ./src

FROM node:20-alpine
WORKDIR /app
COPY --from=builder /app /app

```

```
EXPOSE 4000
CMD ["node", "src/index.js"]
```

This Dockerfile uses a multi-stage build: the first stage installs dependencies and copies source, and the second, lean stage copies only the built output — keeping the final backend image smaller and free of build-time tooling.

8.3 Frontend Dashboard — frontend/Dockerfile

```
FROM node:20-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=build /app/build /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

9. Docker Compose Design

docker-compose.yml orchestrates all six services on a shared internal network, wires environment variables for service discovery, and mounts a volume so PostgreSQL data survives container restarts.

```
version: "3.9"

services:
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: waste_system
      POSTGRES_USER: ${DB_USER}
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - db_data:/var/lib/postgresql/data
      - ./database/init.sql:/docker-entrypoint-initdb.d/init.sql
    networks: [waste_net]

  broker:
    image: eclipse-mosquitto:2
    ports: ["1883:1883"]
    networks: [waste_net]

  ai-model:
    build: ./ai-model
    depends_on: [broker]
    environment:
      MQTT_HOST: broker
    ports: ["8001:8001"]
```

```

networks: [waste_net]

robotic-controller:
  build: ./robotic-controller depends_on:
  [broker, ai-model] environment:
    MQTT_HOST: broker
  networks: [waste_net]

backend:
  build: ./backend depends_on:
  [db, broker] environment:
    DB_HOST: db
    DB_USER:
    ${DB_USER}
    DB_PASSWORD: ${DB_PASSWORD}
    MQTT_HOST: broker
  ports: ["4000:4000"]
  networks: [waste_net]

frontend:
  build: ./frontend depends_on:
  [backend] ports: ["3000:80"]
  networks: [waste_net]

volumes:
  db_data:

networks:
  waste_net:
  driver: bridge

```

9.1 Explanation of Key Elements

Element	Explanation
services	Each of the six services (db, broker, ai-model, robotic-controller, backend, frontend) runs in its own container, built either from an official image or a local Dockerfile.
networks (waste_net)	A shared bridge network lets containers reach each other by service name (e.g. backend connects to 'db', not an IP address).
volumes (db_data)	A named volume persists PostgreSQL data across container restarts and rebuilds.
environment	Injects configuration such as database credentials and the broker host, keeping secrets out of the image itself via a .env file.

Element	Explanation
depends_on	Controls startup order so, for example, the backend does not start before the database and broker are up.
ports	Publishes only the ports that must be reachable from outside the Docker network (frontend on 3000, backend API on 4000, broker on 1883).

10. Container Deployment and Testing

10.1 Running the Application

```
$ cp .env.example .env # fill in DB_USER / DB_PASSWORD
$ docker-compose up --build -d

$ docker ps
CONTAINER ID   IMAGE                                STATUS      PORTS
a1b2c3d4e5f6   waste-segregation-frontend          Up 2 minutes   0.0.0.0:3000->80/tcp
b2c3d4e5f6a1   waste-segregation-backend           Up 2 minutes   0.0.0.0:4000->4000/tcp
c3d4e5f6a1b2   waste-segregation-ai-model          Up 2 minutes   0.0.0.0:8001->8001/tcp
d4e5f6a1b2c3   waste-segregation-robotic-controller Up 2 minutes
e5f6a1b2c3d4   eclipse-mosquitto:2                 Up 2 minutes
0.0.0.0:1883->1883/tcp f6a1b2c3d4e5 postgres:16-alpine Up 2 minutes 5432/tcp
```

10.2 Testing Procedure

- Unit tests: backend and ai-model each run their own test suite (pytest / jest) inside the CI pipeline before the image is built.
- Health checks: curl http://localhost:4000/api/health and http://localhost:8001/health confirm each service is reachable after startup.
- Functional test: a sample waste-item image is POSTed to the ai-model /predict endpoint and the returned category is verified against the expected label.
- Integration test: an end-to-end run publishes a simulated classification event over MQTT and confirms it is persisted by the backend and visible on the dashboard.
- Log inspection: docker-compose logs -f is used to confirm each service starts cleanly with no unhandled exceptions.

```
$ curl -s http://localhost:4000/api/health
{"status":"ok","service":"backend"}

$ curl -s -F "image=@sample_plastic_bottle.jpg" http://localhost:8001/predict
{"category":"plastic","confidence":0.94}

$ docker-compose logs backend --tail=5
backend_1 | [INFO] Connected to database 'waste_system'
backend_1 | [INFO] Subscribed to MQTT topic 'sort/events'
backend_1 | [INFO] Server listening on port 4000
```


This output is treated as evidence of successful execution: each container reports a healthy status, the classification endpoint returns a correct category with high confidence, and the backend logs confirm it received and processed the resulting sort event.

11. Benefits of Git and Docker

Benefit	How It Applies to This Project
Collaboration	Four teams (AI/ML, backend, robotics, frontend) work on isolated feature branches and integrate through pull requests without blocking each other.
Version management	Every change to the classification model, API, or robotic-control logic is tracked, reviewable, and revertible through Git history.
Portability	Docker images run identically on a developer laptop, a CI runner, and the edge server installed at a waste facility.
Deployment consistency	docker-compose reproduces the exact same multi-service environment for every facility rollout, eliminating 'works on my machine' issues.
Scalability	Additional ai-model or robotic-controller replicas can be started independently during peak waste-intake periods.
Maintainability	Clear branch history plus isolated, single-purpose containers make it straightforward to locate, fix, and redeploy a faulty component.

12. Challenges and Improvements

12.1 Challenges Encountered

- Trained model weight files are large binary blobs that bloat the Git repository if committed directly.
- Coordinating merges between the AI-model and backend teams occasionally produced dependency-file conflicts (Section 5.1).
- Providing GPU access inside the ai-model container required extra host configuration (NVIDIA Container Toolkit) not needed by the other services.
- The robotic-controller service depends on physical hardware, which is difficult to fully exercise in an automated CI pipeline.
- Keeping classification latency low enough for a moving conveyor required careful tuning of image size and inference batching.

12.2 Suggested Improvements and Future Enhancements

- Adopt Git LFS or DVC to version large model weights and datasets outside the main Git history.

- Add a CI/CD pipeline (GitHub Actions) that automatically builds, tests, and pushes images to a registry on every merge to develop/main.
 - Introduce a hardware-in-the-loop or simulated-actuator test stage so robotic-controller logic can be validated automatically.
 - Move from Docker Compose to Kubernetes for multi-facility deployments, enabling auto-scaling and rolling updates.
 - Track model versions and accuracy metrics with a tool such as MLflow to support safe rollback of the classifier.
 - Adopt blue-green or canary deployment for the backend and ai-model services to reduce risk during facility upgrades.
-